SYLLABUS - PLAN DE COURS - 2026-2027

Functional Programming with Scala (ADDA74)

Programmes:	Programme grande école ingénieur
Campus:	Villejuif (tous)
Périodes:	Semestre 7

Départements:	Big Data & Machine Learning
Disciplines:	Développement logiciel
Type:	Module
Langue:	Français
Crédits ou coefficient	1
Heures face à face:	30
Heures travail personnel:	15
Total heures:	45

Responsable de module:	Ahmed-ghazi BLAIECH - ahmed-ghazi.blaiech@efrei.fr
	Mohamad GHASSANY - mohamad.ghassany@efrei.fr
	Ahmed-ghazi BLAIECH - ahmed-ghazi.blaiech@efrei.fr
	Mohamad GHASSANY - mohamad.ghassany@efrei.fr
Enseignant:	Nicolas RAYMOND - nicolas.raymond@intervenants.efrei.fr

DESCRIPTION

Résumé

Aperçu du cours
La programmation fonctionnelle est un paradigme où les opérations computationnelles sont modélisées sous forme de fonctions mathématiques. Contrairement au paradigme procédural, le changement d'état est interdit, et la composition et l'application de fonctions servent d'outils principaux. Ce cours introduit les concepts fondamentaux de la programmation fonctionnelle à travers le langage Scala. Les étudiants implémenteront un algorithme MapReduce non distribué pour comprendre sa logique essentielle. Le cours explore également pourquoi Scala est populaire pour les projets d'ingénierie des données, en mettant l'accent sur les avantages de l'immuabilité pour les programmes distribués.

Mots-clés
Programmation fonctionnelle, Scala, fonctions d'ordre supérieur, MapReduce, récursivité, immuabilité, évaluation paresseuse.

Bibliography

- Functional Programming in Scala; Paul Chiusano & Runar Bjarnason; Manning
- https://fr.coursera.org/learn/progfun1 EPFL online materials
- https://www.fun-mooc.fr/courses/parisdiderot/56002S02/session02 Onling course of Paris Diderot

Plan du cours

1. **Introduction et Contexte**
   - Historique et principes de la programmation fonctionnelle
   - Comparaison entre paradigmes procédural, orienté objet, et fonctionnel
   - Applications de la programmation fonctionnelle
2. **Concepts Fondamentaux**
   - Immuabilité : définition, avantages, et applications
   - Fonctions récursives : implémentation et optimisation (tail recursion)
   - Fonctions d'ordre supérieur : map, filter, reduce, et composition fonctionnelle
   - Notions de curryfication et évaluation paresseuse
3. **Concepts Propres à Scala**
   - Collections immuables et mutables dans Scala
   - Traits, case classes et pattern matching
   - Monades : introduction et cas d'utilisation (Option, Either, Future)
4. **Travaux Pratiques et Projet Final**
   - Implémentation d'un algorithme MapReduce non distribué
   - Développement de mini-projets pour consolider les concepts
   - Présentation et soutenance des projets en groupe

Prérequis

- Bon niveau de programmation dans un langage impératif ou fonctionnel
- Familiarité avec l'utilisation et les commandes de shell Unix

Acquis d'apprentissage
À la fin de ce cours, les étudiants seront capables de :

1. Définir et expliquer les concepts fondamentaux de la programmation fonctionnelle.
2. Utiliser les fonctionnalités de Scala pour manipuler des données de manière immuable.
3. Implémenter des algorithmes récursifs et des fonctions d'ordre supérieur.
4. Appliquer les concepts d'immuabilité pour améliorer la robustesse des programmes distribués.
5. Concevoir des programmes fonctionnels efficaces à l'aide de Scala.

COMPETENCES ET INDICATEURS DE DEVELOPPEMENT

Programme grande école ingénieur

B / B - Concevoir des solutions numériques basées sur l'état de l'art

B203 / B203 - Se fixer des objectifs en termes de qualité de service et d'expérience utilisateur

C / C - Mettre en œuvre des solutions numériques

C202 / C202 - Intégrer des composants matériels, logiciels ou des sous-systèmes pertinents dans un système existant ou totalement nouveau

C204 / C204 – Déployer/installer un composant d'une solution numérique dans une configuration donnée

E / E - Œuvrer en mode projet de manière collaborative et responsable, y compris dans un contexte interculturel ou international

E204 / E204 - Organiser son travail selon les workflows, process, rituels, rôles, activités et les outils collaboratifs en vigueur

MATERIEL PEDAGOGIQUE

EPFL online materials

Paul Chiusano & Runar Bjarnason, Functional Programming in Scala, Manning

ACTIVITÉS (À TITRE INDICATIF)

#	Type	Thème ou activité	Durée
1	Cours TP (CTP)		21.00
2	Suivi de projet (PRJ)	Suivi de projet et soutenance	7.00

TRAVAUX & EVALUATIONS

EVALUATIONS

Modalité d'évaluation	Mode	Durée (en heures)	Pondération	Compétences
Global Course Unit --		*	*%	
Devoir écrit (DE) -- Devoir écrit (DE) (Devoir écrit (sur papier))	Individuel	2.00	60.00%	
Projet (PRJ) -- Projet (PRJ) (Présentation de groupe)	Groupe	*	40.00%	

** Intégré dans les activités pédagogiques en face-à-face.*

POLITIQUES

Plagiat

Le plagiat, qu'il soit intentionnel ou non, ne peut être toléré. De ce fait, un travail fourni dans lequel des phrases ou des paragraphes seraient partiellement ou intégralement repris sans référence aux auteurs sera automatiquement sanctionné par la note 0/20 et par une convocation immédiate en conseil de discipline.
Cette disposition s'applique à tout type de texte (livre, article…) ou de contenu numérique issu du web. Les citations sont autorisées, sous réserve que les sources soient clairement indiquées.
L'ensemble de la documentation relative à la démarche anti-plagiat et aux outils Compilatio et Studium est disponible dans l'espace « Direction des études » du portail pédagogique Moodle.